

# Video Person Detection & Recognition Backend

Backend system cho xử lý video với nhận diện và nhận dạng người sử dụng YOLO8 và ViT-DINO.

## Tính năng chính

### Luồng xử lý (theo Guide.md)

1. **Nhận video** - Upload file hoặc từ URL
2. **Tách frame** - Trích xuất 2 FPS
3. **Xử lý ảnh nâng cao:**
  - Khử nhiễu (Gaussian blur, bilateral filter)
  - Tăng cường độ tương phản (CLAHE)
  - Cân bằng sáng (xử lý ảnh tối/sáng)
  - Làm nét ảnh (Sharpening)
4. **Nhận diện người** - YOLO8 trained model
5. **Lưu trữ detections** - MongoDB với metadata đầy đủ
6. **Trích xuất đặc trưng** - ViT với DINO
7. **So khớp người quen** - Cosine Similarity ( $>0.7$ )
8. **Trả về kết quả** - JSON với tất cả thông tin

## Cấu trúc dự án

```
video-detection-backend/  
├── main.py           # FastAPI application  
├── video_processor.py # Video processing pipeline  
├── image_enhancement.py # Image enhancement module  
├── feature_extractor.py # ViT feature extraction  
├── database.py       # MongoDB operations  
├── models.py         # Pydantic schemas  
├── requirements.txt   # Python dependencies  
├── Dockerfile        # Docker configuration  
├── docker-compose.yml # Docker compose setup  
├── .env.example       # Environment variables template  
├── uploads/           # Video uploads  
├── output/  
│   └── detected/      # Detected person frames
```

```
|— temp/          # Temporary files
|— models/       # ML model files
   |— yolov8_person_detection.pt
   |— vit_dino_feature_extractor.pt
```

## Cài đặt

### Yêu cầu

- Python 3.10+
- MongoDB 7.0+
- CUDA (tùy chọn, cho GPU acceleration)
- Docker & Docker Compose (tùy chọn)

### Cài đặt thủ công

#### 1. Clone repository

```
bash

git clone <repository-url>
cd video-detection-backend
```

#### 2. Tạo virtual environment

```
bash

python -m venv venv
source venv/bin/activate # Linux/Mac
# hoặc
venv\Scripts\activate # Windows
```

#### 3. Cài đặt dependencies

```
bash

pip install -r requirements.txt
```

#### 4. Cấu hình environment

```
bash
```

```
cp .env.example .env
```

```
# Chỉnh sửa .env với cấu hình của bạn
```

## 5. Đặt model files

```
bash
```

```
# Đặt trained models vào thư mục models/
```

```
models/
```

```
├── yolov8_person_detection.pt
```

```
└── vit_dino_feature_extractor.pt
```

## 6. Chạy MongoDB

```
bash
```

```
# Sử dụng Docker
```

```
docker run -d -p 27017:27017 \
```

```
-e MONGO_INITDB_ROOT_USERNAME=admin \
```

```
-e MONGO_INITDB_ROOT_PASSWORD=admin123 \
```

```
mongo:7.0
```

## 7. Chạy application

```
bash
```

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

## Cài đặt với Docker

```
bash
```

*# Build và chạy tất cả services*

`docker-compose up -d`

*# Xem logs*

`docker-compose logs -f backend`

*# Dừng services*

`docker-compose down`

## API Endpoints

### 1. Process Video (Upload)

bash

POST /process/upload

Content-Type: multipart/form-data

```
curl -X POST "http://localhost:8000/process/upload" \  
-F "file=@video.mp4"
```

### 2. Process Video (URL)

bash

POST /process/url

Content-Type: application/json

```
curl -X POST "http://localhost:8000/process/url" \  
-H "Content-Type: application/json" \  
-d '{  
  "url": "https://example.com/video.mp4",  
  "video_name": "test_video"  
}
```

### 3. Get Results

bash

GET /results/{job\_id}

```
curl "http://localhost:8000/results/{job_id}"
```

## 4. Get Detections

bash

GET /detections/{video\_id}?skip=0&limit=100

curl "http://localhost:8000/detections/{video\_id}"

## 5. Add Known Person

bash

POST /features/add

Content-Type: application/json

```
curl -X POST "http://localhost:8000/features/add" \  
-H "Content-Type: application/json" \  
-d '{  
  "person_id": "person_001",  
  "name": "Nguyen Van A",  
  "feature_vector": [0.123, 0.456, ...]  
}'
```

## 6. List Known Persons

bash

GET /features/list

curl "http://localhost:8000/features/list"

## Database Schema

### Collections

#### 1. detections

json

```
{
  "detection_id": "uuid",
  "video_id": "uuid",
  "video_name": "string",
  "frame_number": 120,
  "timestamp": 4.5,
  "bounding_box": {
    "x1": 100, "y1": 200,
    "x2": 300, "y2": 500,
    "width": 200, "height": 300
  },
  "confidence": 0.95,
  "feature_vector": [0.123, ...],
  "saved_frame_path": "path/to/frame.jpg",
  "is_known": true,
  "matched_person_id": "person_001",
  "matched_person_name": "Nguyen Van A",
  "similarity_score": 0.85,
  "detected_at": "2024-01-01T12:00:00"
}
```

## 2. known\_persons

json

```
{
  "person_id": "person_001",
  "name": "Nguyen Van A",
  "feature_vector": [0.123, ...],
  "added_at": "2024-01-01T12:00:00",
  "updated_at": "2024-01-01T12:00:00"
}
```

## 3. processing\_results

json

```
{
  "job_id": "uuid",
  "video_id": "uuid",
  "video_name": "string",
  "processed_at": "2024-01-01T12:00:00",
  "total_frames": 1000,
  "processed_frames": 1000,
  "extracted_frames": 500,
  "total_detections": 50,
  "known_persons": 30,
  "unknown_persons": 20,
  "detections": [...]
}
```

## Testing

### Test với Python

```
python

import requests

# Upload video
files = {'file': open('test_video.mp4', 'rb')}
response = requests.post('http://localhost:8000/process/upload', files=files)
result = response.json()

print(f'Job ID: {result['job_id']}')
print(f'Total detections: {result['results']['total_detections']}')
```

### Test với curl

```
bash

# Health check
curl http://localhost:8000/health

# Process video from URL
curl -X POST "http://localhost:8000/process/url" \
  -H "Content-Type: application/json" \
  -d '{"url": "https://example.com/video.mp4"}'
```

## Configuration

### Tùy chỉnh ngưỡng

Trong `video_processor.py`:

```
python

self.confidence_threshold = 0.5 # YOLO confidence
self.known_person_threshold = 0.7 # Similarity threshold
```

### Tùy chỉnh frame rate

```
python

# Trong process_video(): 2 FPS
frame_interval = int(fps / 2)
```

### Tùy chỉnh image enhancement

Trong `image_enhancement.py`, điều chỉnh parameters:

```
python

self.clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
```

## Troubleshooting

### MongoDB connection failed

```
bash

# Kiểm tra MongoDB running
docker ps | grep mongo

# Kiểm tra connection string trong .env
MONGODB_URL=mongodb://admin:admin123@localhost:27017/
```

### CUDA/GPU issues

```
bash
```

```
# Kiểm tra CUDA available
```

```
python -c "import torch; print(torch.cuda.is_available())"
```

```
# Nếu không có GPU, model sẽ tự động chạy trên CPU
```

## Model not found

```
bash
```

```
# Đảm bảo model files tồn tại
```

```
ls -la models/
```

```
# Hoặc update paths trong code
```

## Performance

- **Frame extraction:** ~2 FPS (configurable)
- **YOLO detection:** ~30-50 FPS (GPU) / ~5-10 FPS (CPU)
- **ViT feature extraction:** ~20-30 FPS (GPU) / ~2-5 FPS (CPU)
- **MongoDB write:** ~1000 documents/second

## Security

- API không có authentication (thêm JWT nếu cần)
- MongoDB credentials trong docker-compose
- File uploads giới hạn size (cấu hình trong FastAPI)

## License

MIT License

## Contributors

Your Team

## Acknowledgments

- YOLOv8 by Ultralytics
- DINO by Facebook Research

- FastAPI framework
- MongoDB Motor driver